

# R Lab Week 2

Robert Gulotty

9/30/2016

---

## Vectorized code with dplyr, joins

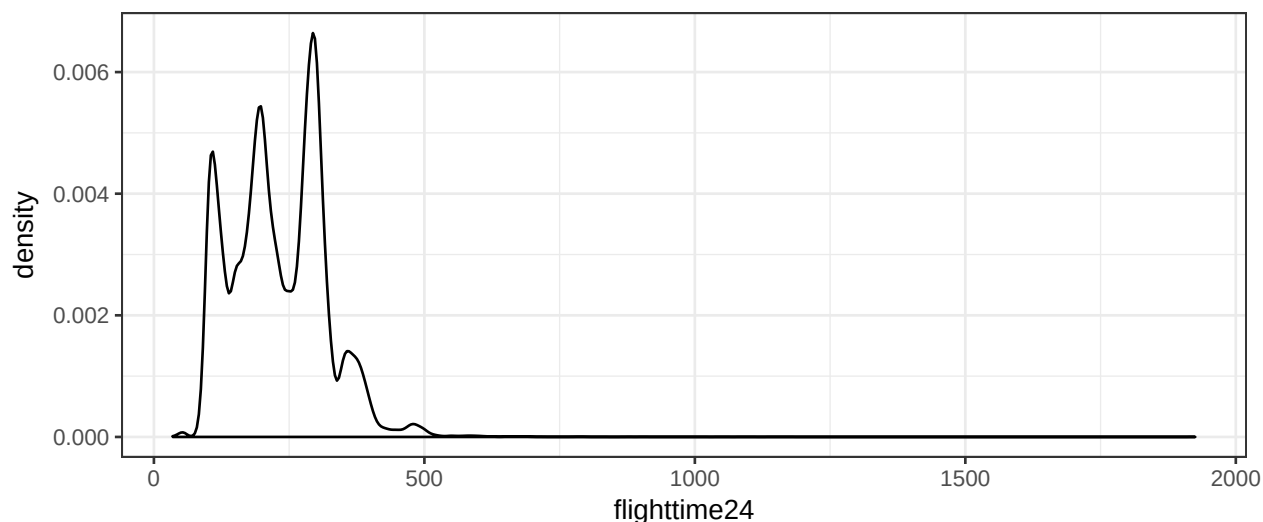
This is the last we will see of the “flights” data. First, recall the following operations:

```
flightscomparison <- mutate(flights,
  flighttime = arr_time - dep_time,
  flighttime24 = ifelse( flighttime < 0, flighttime+2400, flighttime),
  ChristmasEve = ifelse( month==12 & day==24 & dep_time>1900,
    "Christmas Eve", "Un-Christmas Eve"))
```

You may have noticed the one hour peaks in the density for the flights plots, such as the following.

```
ggplot(flightscomparison, aes(flighttime24))+geom_density()+theme_bw()
```

```
## Warning: Removed 8713 rows containing non-finite values (stat_density).
```



One of the things that is going on here is the time zone changes. Luckily, there is a dataset of the location of each of the airports in the nycflights13 package, called `airports`.

To combine the datasets, we need a column that is the same across each dataset.

```
str(airports)
```

```
## Classes 'tbl_df', 'tbl' and 'data.frame':   1458 obs. of  8 variables:
## $ faa   : chr  "04G" "06A" "06C" "06N" ...
## $ name  : chr  "Lansdowne Airport" "Moton Field Municipal Airport" "Schaumburg Regional" "Randall Ai
## $ lat   : num  41.1 32.5 42 41.4 31.1 ...
## $ lon   : num  -80.6 -85.7 -88.1 -74.4 -81.4 ...
## $ alt   : int  1044 264 801 523 11 1593 730 492 1000 108 ...
## $ tz    : num  -5 -6 -6 -5 -5 -5 -5 -5 -8 ...
## $ dst   : chr  "A" "A" "A" "A" ...
## $ tzone : chr  "America/New_York" "America/Chicago" "America/Chicago" "America/New_York" ...
## - attr(*, "spec")=List of 2
## ..$ cols   :List of 12
## .. ..$ id    : list()
## .. ..$- attr(*, "class")= chr  "collector_integer" "collector"
## .. ..$ name   : list()
## .. ..$- attr(*, "class")= chr  "collector_character" "collector"
## .. ..$ city   : list()
## .. ..$- attr(*, "class")= chr  "collector_character" "collector"
## .. ..$ country: list()
## .. ..$- attr(*, "class")= chr  "collector_character" "collector"
## .. ..$ faa    : list()
## .. ..$- attr(*, "class")= chr  "collector_character" "collector"
## .. ..$ icao    : list()
## .. ..$- attr(*, "class")= chr  "collector_character" "collector"
## .. ..$ lat    : list()
## .. ..$- attr(*, "class")= chr  "collector_double" "collector"
## .. ..$ lon    : list()
## .. ..$- attr(*, "class")= chr  "collector_double" "collector"
## .. ..$ alt    : list()
## .. ..$- attr(*, "class")= chr  "collector_integer" "collector"
## .. ..$ tz     : list()
## .. ..$- attr(*, "class")= chr  "collector_double" "collector"
## .. ..$ dst    : list()
## .. ..$- attr(*, "class")= chr  "collector_character" "collector"
## .. ..$ tzone  : list()
## .. ..$- attr(*, "class")= chr  "collector_character" "collector"
## ..$ default: list()
## .. ..$- attr(*, "class")= chr  "collector_guess" "collector"
## ..$- attr(*, "class")= chr "col_spec"
```

```
str(flightscomparison)
```

```
## Classes 'tbl_df', 'tbl' and 'data.frame':   336776 obs. of  22 variables:
## $ year      : int  2013 2013 2013 2013 2013 2013 2013 2013 2013 2013 ...
## $ month     : int  1 1 1 1 1 1 1 1 1 1 ...
## $ day       : int  1 1 1 1 1 1 1 1 1 1 ...
## $ dep_time  : int  517 533 542 544 554 554 555 557 557 558 ...
## $ sched_dep_time: int  515 529 540 545 600 558 600 600 600 600 ...
## $ dep_delay : num  2 4 2 -1 -6 -4 -5 -3 -3 -2 ...
## $ arr_time  : int  830 850 923 1004 812 740 913 709 838 753 ...
## $ sched_arr_time: int  819 830 850 1022 837 728 854 723 846 745 ...
## $ arr_delay : num  11 20 33 -18 -25 12 19 -14 -8 8 ...
```

```
## $ carrier      : chr  "UA" "UA" "AA" "B6" ...
## $ flight       : int   1545 1714 1141 725 461 1696 507 5708 79 301 ...
## $ tailnum      : chr   "N14228" "N24211" "N619AA" "N804JB" ...
## $ origin       : chr   "EWR" "LGA" "JFK" "JFK" ...
## $ dest         : chr   "IAH" "IAH" "MIA" "BQN" ...
## $ air_time     : num    227 227 160 183 116 150 158 53 140 138 ...
## $ distance     : num    1400 1416 1089 1576 762 ...
## $ hour         : num     5 5 5 5 6 5 6 6 6 6 ...
## $ minute       : num    15 29 40 45 0 58 0 0 0 0 ...
## $ time_hour    : POSIXct, format: "2013-01-01 05:00:00" "2013-01-01 05:00:00" ...
## $ flighttime   : int    313 317 381 460 258 186 358 152 281 195 ...
## $ flighttime24 : num    313 317 381 460 258 186 358 152 281 195 ...
## $ ChristmasEve : chr    "Un-Christmas Eve" "Un-Christmas Eve" "Un-Christmas Eve" "Un-Christmas Eve"
```

It looks like the `faa` column of `airports` is similar to that of the `origin` and `dest` columns of `flightscomparison`. Lets join on the “dest” column.

## Intro to pipes

In order to combine the two dataframes, we need a column in each dataframe with the same name. To accomplish this, we add a column to airports using the “mutate” command. I am going to start using the pipe `%>%` commands now.

```
airports <- airports %>% mutate( dest = faa)
```

Here we pass the thing on the left to the first argument in the next function. You will see that it makes code much nicer. Consider the following:

```
airportsTZ4 <- filter(airports, tz == -5)
```

```
airportsTZ4 <- mutate(airportsTZ5, swardist = sqrt((lat - 40.7)^2+(lon-74)^2))
```

```
ggplot(airportsTZ5, aes(swardist, alt))+geom_point()
```

Notice how many times airportsTZ5 is written. We can use the `%>%` operator to save typing.

```
airports %>%
```

```
  filter( tz == -5) %>%
```

```
  mutate( swardist = sqrt((lat - 40.7)^2+(lon-74)^2)) %>%
```

```
  ggplot(aes(swardist, alt))+geom_point()
```

## Intro to join

One of the most useful commands in R is the join command. Here we will use `left_join`, which keeps all the data in the original dataset, and matches the rows it can find in the right.

```
flightsandairports <- flightscomparison%>%left_join(airports)
```

```
## Joining, by = "dest"
```

Notice, that this offers a nice warning to tell you what columns it used to join, but it does not tell you how many rows of airports it dropped to make the final dataset. To do so, we can use the `anti_join`.

```
Administrations <- data.frame(years=2001:2010, Presidents=c(rep("R", 8), rep("D",2)))
Troops <- data.frame(years=c(2000,2001:2004, 2006,2007, 2009,2010,2012),
                     troops=c(0,2500,9700,13100,20300,20300,25000,67000, 100000, 77000))
AdminTroops<- Administrations%>%left_join(Troops)
```

```
## Joining, by = "years"
```

```
missingTroops<-Administrations%>%anti_join(Troops)
```

```
## Joining, by = "years"
```

```
missingAdmin<-Troops%>%anti_join(Administrations)
```

```
## Joining, by = "years"
```

```
missingTroops
```

```
##   years Presidents
## 1  2005           R
## 2  2008           R
```

```
missingAdmin
```

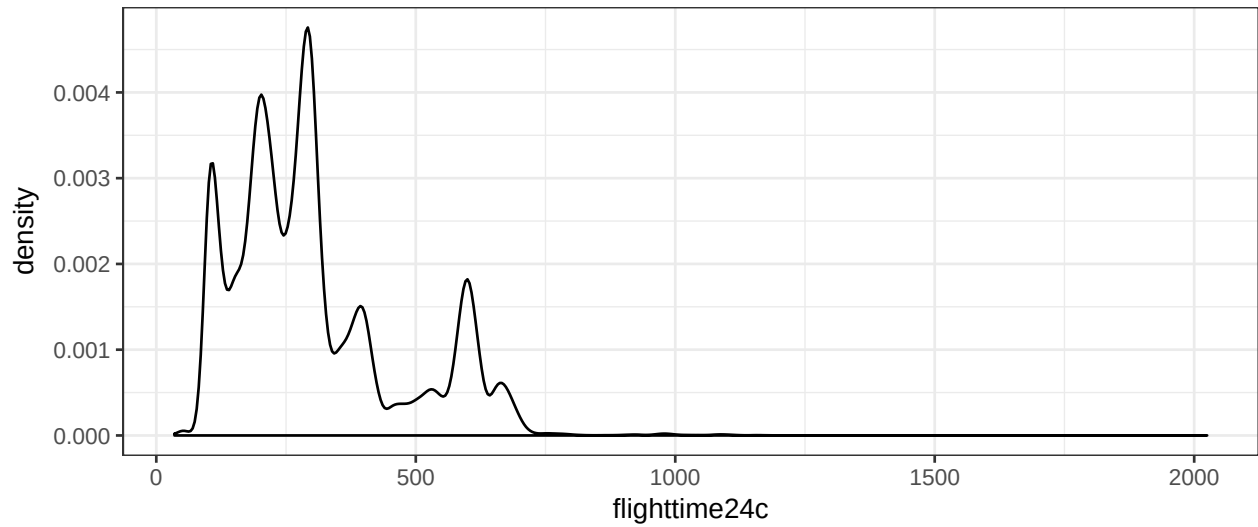
```
##   years troops
## 1  2000      0
## 2  2012 77000
```

Great, now we can the plot, but correct for the timezone.

```
flightsandairports <- flightsandairports%>%mutate(flighttime24c=flighttime24 - 500 - tz*100)

ggplot(flightsandairports, aes(flighttime24c))+geom_density()+theme_bw()
```

## Warning: Removed 16273 rows containing non-finite values (stat\_density).



## Join Examples

First, make two datasets using the `data.frame` command. You do so by listing the names of the columns, and here we are manually entering data from the huffington post estimates of a democratic victory as well as the electoral college count of each state. Then we are listing the fivethirtyeight tipping point states, as well as the betting market probabilities at the same point a few weeks before election day. We also set the `stringsAsFactors` setting to `FALSE`.

```
allstates<-data.frame(state_name=state.name,
electoral_c_count=c(9L, 3, 11, 6, 55, 9, 7, 3, 29,
                    16, 4, 4, 20,11, 6, 6, 8, 8,
                    4, 10, 11, 16, 10, 6, 10, 3,
                    5,6, 4, 14, 5, 29, 15, 3, 18,
                    7, 7, 20, 4, 9, 3,11, 38, 6,
                    3, 13, 12, 5, 10, 3),
huffpowin=c(0, 0, 0.276, 0, 1, 0.973, 0.995, 0.999, 0.835,
            0.086, 1, 0, 1, 0, 0.216, 0.001, 0,
            0, 0.954, 1, 1, 0.887, 0.959, 0.008,
            0.021, 0, 0, 0.574, 0.928, 0.999, 0.999, 1,
            0.684, 0, 0.573,0, 0.997, 0.95, 1, 0.046,
            0, 0, 0.014, 0, 1, 0.976, 0.999, 0, 0.833, 0),
stringsAsFactors=FALSE)

tippingstates <- data.frame( state_name = c("Florida", "Pennsylvania", "North Carolina",
"Michigan", "Ohio", "Wisconsin", "Colorado", "Virginia", "Minnesota", "Nevada"),
    betting=c(80, 86, 82, 90,76,88, 86,90,90,81),
    stringsAsFactors=FALSE)
```

- `left_join` keeps the left hand data

```
allstateswithtip <- left_join(allstates, tippingstates)
```

```
## Joining, by = "state_name"
```

- `full_join` keeps everything

```
allstatesandtip <- full_join(allstates, tippingstates)
```

```
## Joining, by = "state_name"
```

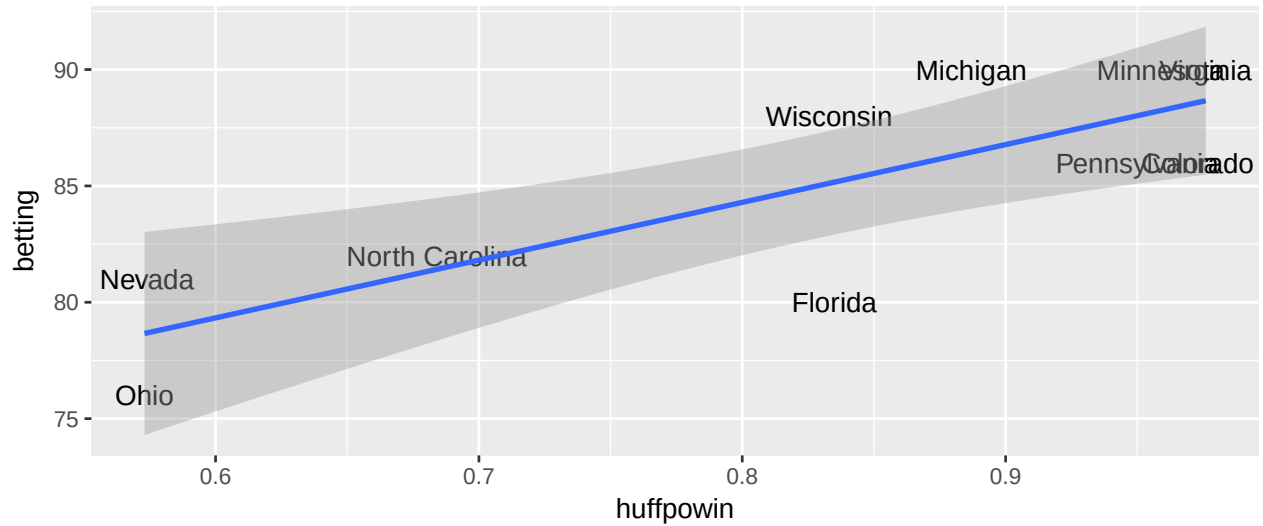
- `inner_join` keeps only exact matches

```
onlytip <- inner_join(allstates, tippingstates)
```

```
## Joining, by = "state_name"
```

Lets see which market is most overconfident:

```
ggplot(onlytip, aes(huffpowin, betting))+  
  geom_text(aes(label=state_name))+  
  geom_smooth(method="lm")
```



Seems that the market gave the democrats a much higher chance of winning Michigan and Wisconsin than the polls did, and a much lower chance of winning Florida.



## Intro to vectorization.

Vectorization means using a function rather than a loop. The main idea is that R works better when it is not looping. For example, with the states, we could change the huffpowin variable to a percentage by looping through the rows and multiplying by 100.

```
onlytip$huffpowinperc <- NA

for (i in 1:length(onlytip$huffpowin)) {
  onlytip$huffpowinperc[i] <- onlytip$huffpowin[i] * 100
}
```

However, we could just use the mutate command to transform it all at once.

```
onlytip <- mutate(onlytip, huffpowinperc= huffpowin*100)
```

In general there may be a tradeoff between the ease of writing a command and the speed that the computer can calculate the answer, but in this case there are also readability advantages to using the vectorized code.

## World Religion Dataset

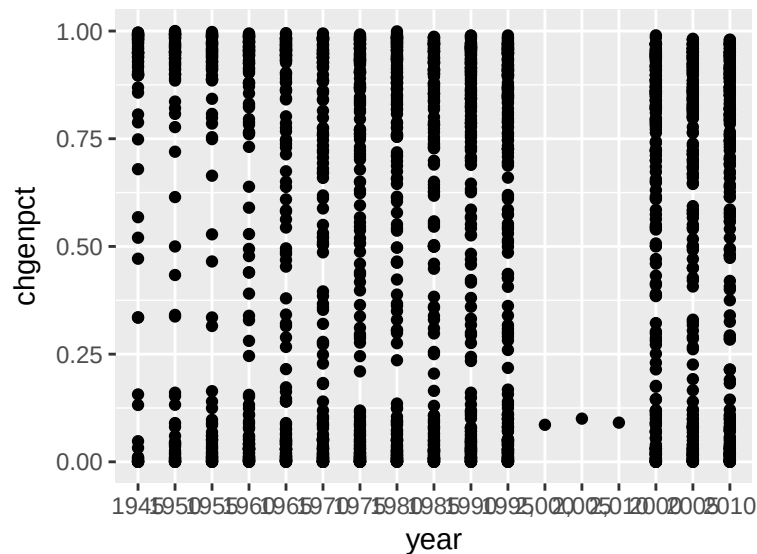
For the remainder of the excersize, we will be playing with the World Religion Dataset, which is in STATA format. We can read STATA format with the `haven` library:

```
library(haven)
```

```
religions<-read_dta("~/Dropbox/mathCamp/Code/data/WorldReligion.DTA")
```

Lets play with the “religion” dataset. Here we have the percent of Christians (All denominations) by year.

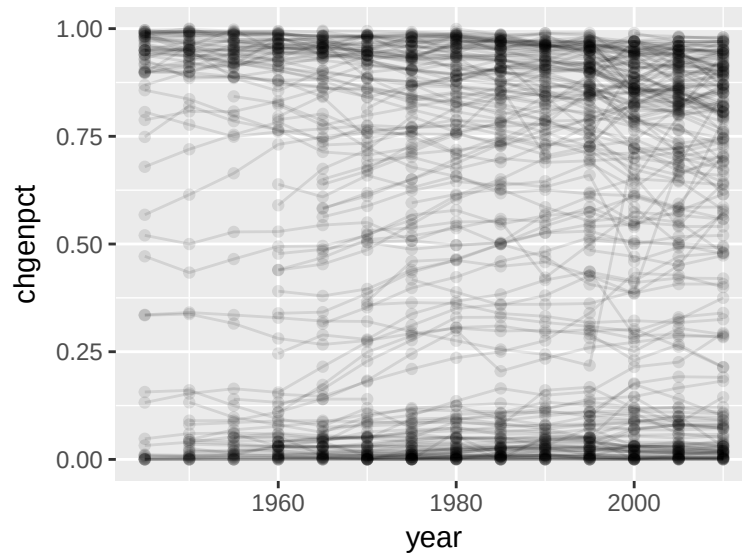
```
ggplot(religions, aes(year,chgenpct))+geom_point()
```



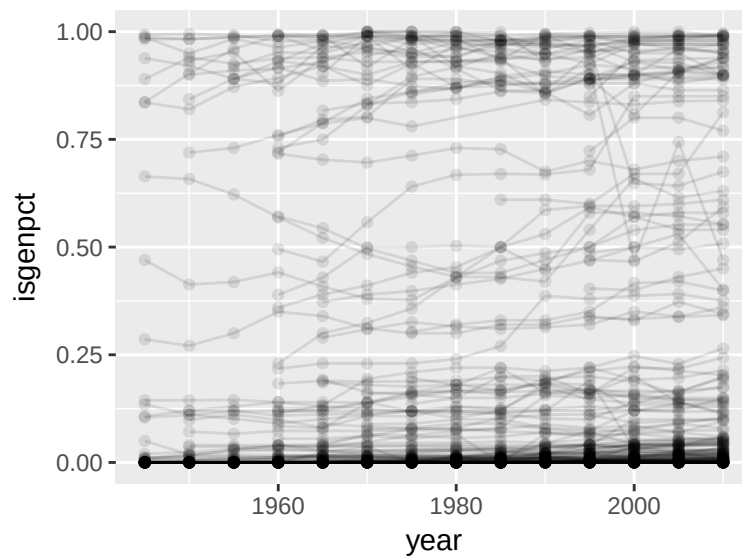
I notice that the years are actually being treated like categorical data, because of some commas in the year variable. Get rid of the commas in the year variable, and tranform it into a numeric data type. We do so with the `gsub()` command.

```
religions <- religions %>% mutate(year = as.numeric(gsub(",", "", year)))

ggplot(religions, aes(year, chgenpct, group = country)) + geom_point(alpha =
.1) + geom_line(alpha = .1)
```



```
ggplot(religions, aes(year, isgenpct, group = country)) + geom_point(alpha =
.1) + geom_line(alpha = .1)
```



Ok, now we can try to isolate those countries that have a big change. To do so, we group\_by country, and then summarise. This allows us to do very advanced calculations by group, which we will learn about in the coming weeks.

```
religionchange <- religions %>%  
  group_by(country) %>%  
  summarise(maxinc = max(c(diff(isgenpct), 0), na.rm = T),  
    maxdec = min(c(diff(isgenpct), 0), na.rm = T)) %>%  
  arrange(maxdec)  
  
religionchange <- religionchange %>%  
  mutate(bigchange = ifelse(maxinc > .1 | maxdec < -.1, 1, 0))
```

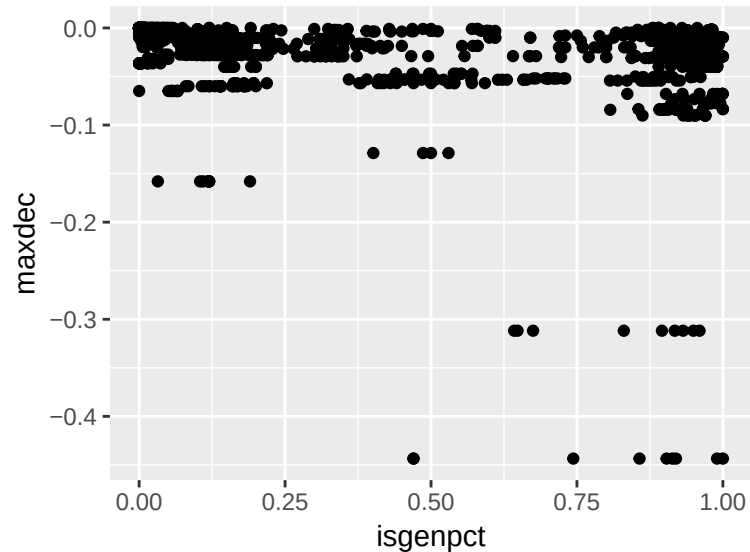
Ok, now we recombine this with the original dataset so that we can use it as a variable in the plots:

```
religions<-religions%>%left_join(religionchange)
```

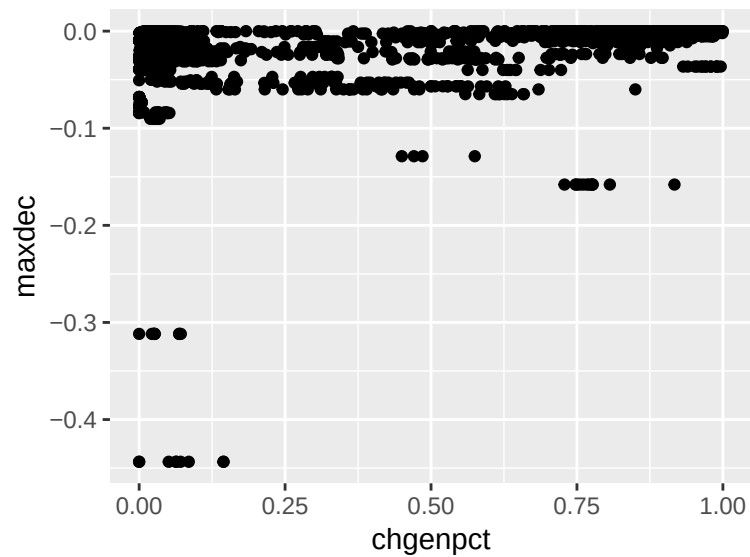
```
## Joining, by = "country"
```

Here we plot the extent that a country's respondents report to follow any form of islam and the maximum change in the whole dataset, and the same for Christians:

```
ggplot(religions, aes(isgenpct, maxdec, group = "country")) + geom_point()
```



```
ggplot(religions, aes(chgenpct, maxdec, group = "country")) + geom_point()
```



I notice a few outliers, which we can isolate using the filter command, followed by a select command, which identifies columns

```
religions%>%filter(maxdec< -.1)%>%select(year, country, reliab)
```

```
## # A tibble: 34 x 3
##   year    country    reliab
##   <dbl>    <chr> <dbl+lbl>
## 1  1945 Yugoslvaia      2
## 2  1950 Yugoslvaia      2
## 3  1955 Yugoslvaia      2
## 4  1960 Yugoslvaia      1
## 5  1965 Yugoslvaia      1
## 6  1970 Yugoslvaia      2
## 7  1975 Yugoslvaia      1
## 8  1980 Yugoslvaia      2
## 9  1985 Yugoslvaia      2
## 10 1990 Yugoslvaia      2
## # ... with 24 more rows
```

One thing I noticed was that on the right hand side, we have an indicator for the reliability of the data. It is categorical. Very often we have two categorical data and it is difficult to visualize it. One way to do so is in a table:

```
table(religions$reliab, religions$bigchange)
```

```
##
##      0      1
## 1  465    26
## 2 1196    54
## 3  248     6
```

Tables are nice, but what we are really looking for is the relative percentages. Is the big change coming along with unreliable data? What we can do is wrap the table in a `prop.table`, which essentially divides by the total of the rows (if we set `margin=1`), or the columns (if we set `margin = 2`).

```
prop.table(table(religions$reliab, religions$bigchange), margin=1)
```

```
##
##      0      1
## 1 0.94704684 0.05295316
## 2 0.95680000 0.04320000
## 3 0.97637795 0.02362205
```

Whenever reporting such a table, it is best to round the results, and or multiply by 100.

```
round(prop.table(table(religions$reliab, religions$bigchange), margin=1),3)
```

```
##
##      0      1
## 1 0.947 0.053
## 2 0.957 0.043
## 3 0.976 0.024
```

If you want the `prop.table` to be nicely printed, one option is to use `xtable`, and write the object to html.

```
library(xtable)
```

```
out.table <-
```

```
xtable(prop.table(table(religions$reliab, religions$bigchange), margin = 1), digits = 2)
print(out.table, type = "html", file = "~/Dropbox/mathCamp/Code/data/religiousreliability.html")
```

There is also the excellent stargazer package.

```
library(stargazer)
stargazer(out.table, summary = F, type = "text")
```

## Intro to Regression

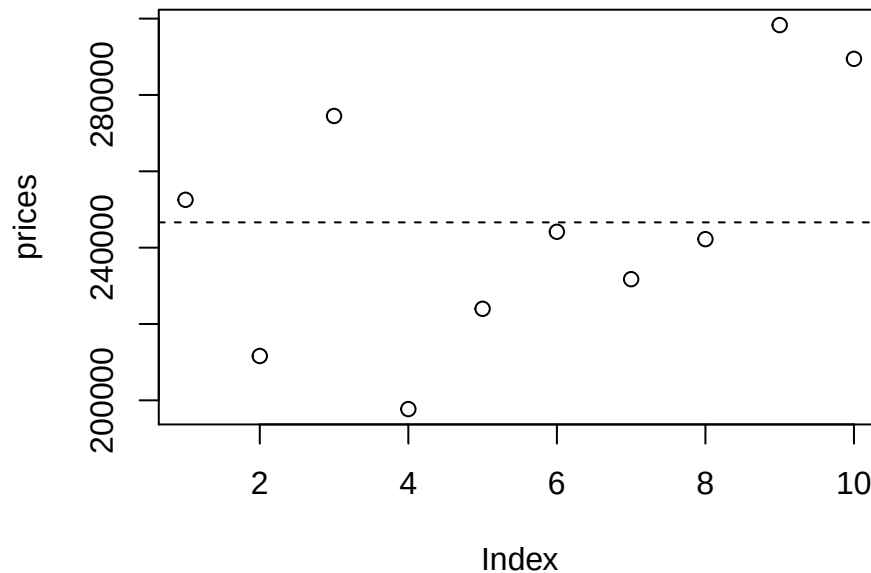
In addition to making a table, we could fit a regression to the data, here we use the “probit” regression, but first, a little background.

When we want to predict the value of something, say Condominium prices, given a set of data, say 10 past purchases, we can use the average of the past prices as our prediction of the future price. Our best guess is the average.

$$E(\text{price}) = \alpha$$

For example, consider the following:

```
prices <- c(252539, 211620, 274491, 197712, 223979, 244161,
           231741, 242225, 298309, 289457)
plot(prices)
abline(h = mean(prices), lty=2)
```



Our best guess for some new point is just the average. In linear model terminology, this would be a model with just the intercept.

```
mean(prices)
```

```
## [1] 246623.4
```

```
summary(lm(prices ~ 1))
```

```
##
## Call:
## lm(formula = prices ~ 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -48911 -20704  -3430   22380  51686
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   246623     10379    23.76 1.98e-09 ***
```



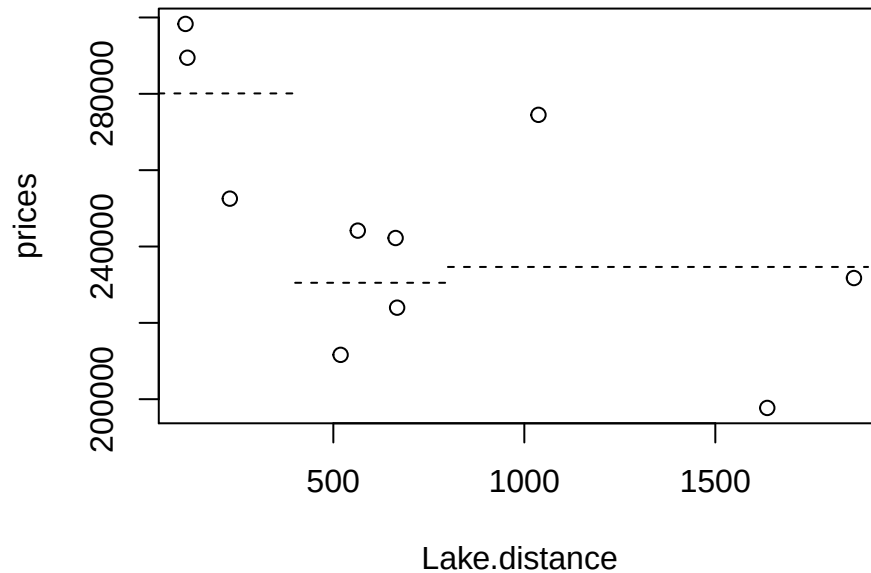
```
## ---  
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
##  
## Residual standard error: 32820 on 9 degrees of freedom
```

Now, suppose that we learn something new, and that is that the Condos are a certain distance from the lake. Now perhaps we want to take advantage of this information, and use the average in three “bins”. Here we take the average for lake distances between 0 and 400, between 400 and 800, and above

```
Lake.distance <- c(229, 519, 1037, 1636, 667, 564, 1863, 663, 113, 118)
```

```
plot(Lake.distance,prices)
```

```
segments(x0=c(0,400,800), x1=c(400,800, 1900), y0=c(mean(prices[Lake.distance<400]), mean(prices[Lake.d
```



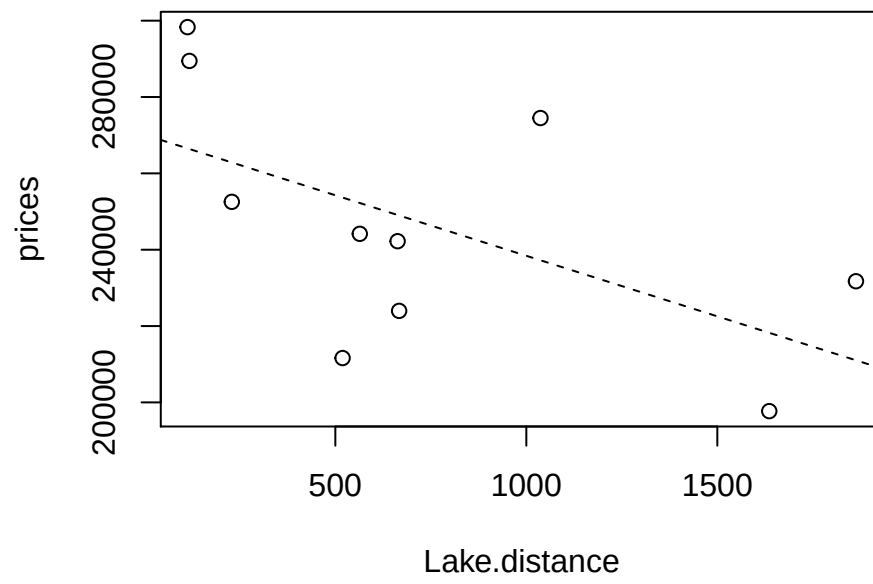
Pretty good, but it seems that the last bin is driven a bit by one point. We might expect that the price is some linear function of the lake distance, a model which adds predictive power. To do so, let's model price as a linear function, with both a slope and an intercept. The intercept is the price when the lake distance is 0.

$$E(\text{price}|\text{lake.distance}) = \alpha + \beta \text{lake.distance}$$

```
Lake.distance <- c(229, 519, 1037, 1636, 667, 564, 1863, 663, 113, 118)
```

```
plot(Lake.distance,prices)
```

```
abline(lm(prices~Lake.distance), lty=2)
```



A regression is a statistical model which estimates the relationship between one variable, called the *dependent variable* and one or more other variables, called the *independent variables*. Regressions require writing down a model, which relates these independent variables to the dependent variable in a formula that describes both the observable and systematic part of the relationship, and the unobservable ideosyncratic component. If we call an observation of the dependent variable  $y_i$ , and the observation of the independent variable  $x_i$ , we can say they are related in the following linear model.

$$y_i = \alpha + \beta x_i + \epsilon_i$$

Where  $\alpha$  and  $\beta$  are called parameters. These are systematic, and apply for all observations.  $\epsilon$  is ideosyncratic, and is unique to each individual. The purpose of the regression model is to estimate  $\alpha$  and  $\beta$ , either to make predictions, or to discover the amount of  $y$  that we should expect a change in  $x$  to produce:

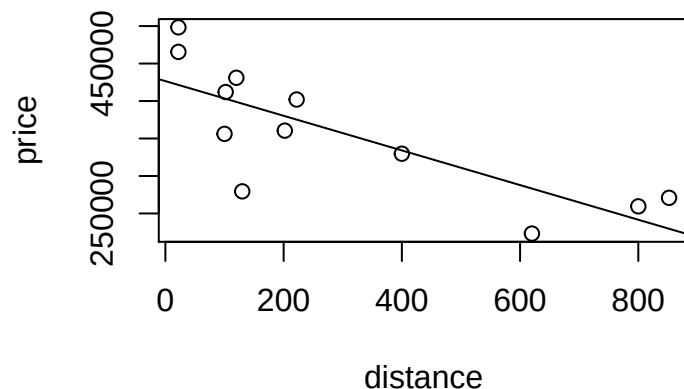
$$\begin{aligned}\frac{dy}{dx} &= \frac{d}{dx}[\alpha + \beta x_i + \epsilon_i] \\ \frac{dy}{dx} &= \frac{d}{dx}\alpha + \beta \frac{d}{dx}x_i + \frac{d}{dx}\epsilon_i\end{aligned}$$

$$\frac{dy}{dx} = \beta$$

Which is consistent with  $\beta$  being the slope of the line produced by the regression. The standard regression practice is to assume a distribution for  $\epsilon$ , usually  $\epsilon \sim N(0, \sigma^2)$ . This and the other assumptions of regression will be covered in the following course. For now, there is one that makes the standard linear model inappropriate for our purposes, there is an assumption that the relationship is *linear*.

For example, suppose we get another sample of Condos. We want to predict the value of condo in the South Loop, and we observe the distance to the # 2 bus in feet, and the price of each condo.

```
distance<- c( 22, 120, 400 ,102,22, 620, 100 ,202,222, 130, 800 ,852)
price<-c(465600, 431000, 329700, 412000, 498300, 223200,
        356200, 360500, 402000, 279500, 259500, 270900)
modcondo<-lm(price~distance)
plot(distance, price)
abline(modcondo)
```



| <i>Dependent variable:</i> |                        |
|----------------------------|------------------------|
|                            | price                  |
| distance                   | -231***<br>(57)        |
| Constant                   | 426,554***<br>(23,524) |
| Observations               | 12                     |
| R <sup>2</sup>             | 0.623                  |
| Adjusted R <sup>2</sup>    | 0.585                  |
| Residual Std. Error        | 56,239 (df = 10)       |
| F Statistic                | 16.517*** (df = 1; 10) |

*Note:* \*p<0.1; \*\*p<0.05; \*\*\*p<0.01

This looks like for every foot that the condo is away from the bus stop, the price declines by about \$231. Our best guess for the price of a condo that is 600 feet away from the bus is about  $426000 - 231 \times 600$

Suppose, rather than the price, we just had a categorical outcome, like the bigchange variable from above? Lets say rather than the price, we just had an indicator for whether my real estate agent would show it to me, after being told that I need to use the bus. Lets say that we have a 1 if the real estate agent shows it, and a 0 if he does not. Here I show both the linear model, as well as the prediction from that linear model if the line falls above .5 or below .5.

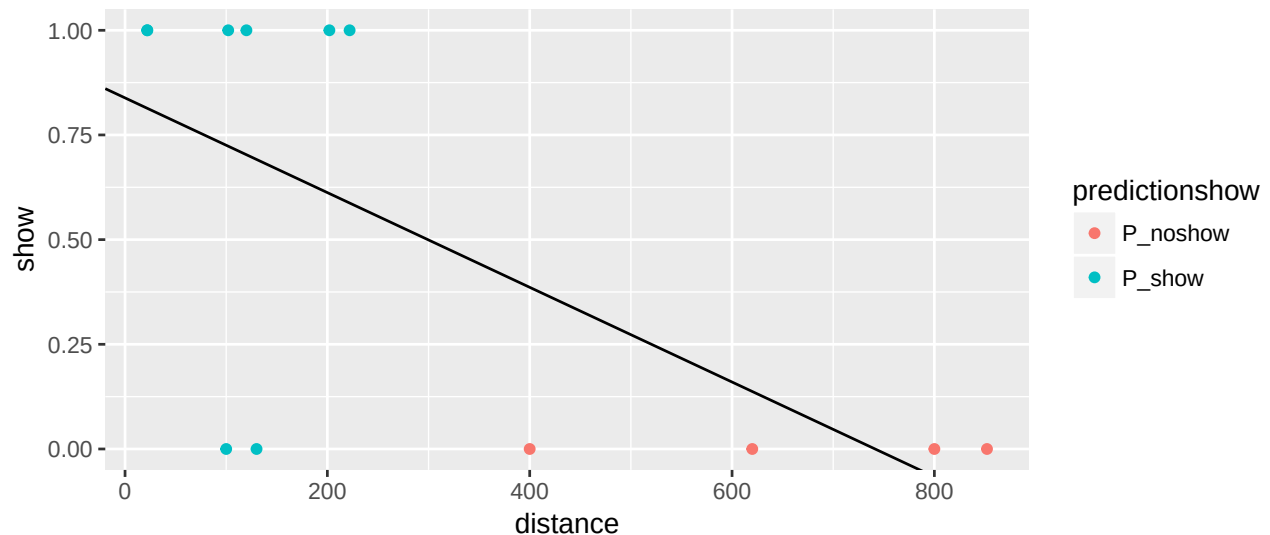
```
distance<- c( 22, 120, 400 ,102,22, 620, 100 ,202,222, 130, 800 ,852)

RealEstateAgent <- c("show", "show", "don't show", "show", "show",
                     "don't show", "don't show", "show", "show",
                     "don't show", "don't show", "don't show")

show <- ifelse(RealEstateAgent=="show", 1 , 0)

modshow <- lm(show~distance)
modpredictions <- data.frame(distance, show,
                              predictionshow= ifelse(predict(modshow)>.5, "P_show","P_noshow"))

coefmodshow <- coef(modshow)
ggplot(modpredictions, aes(distance, show, color=predictionshow) )+
  geom_point()+
  geom_abline(intercept=coefmodshow[1], slope=coefmodshow[2])
```



This model works just fine, but the misses above and below the line are quite uneven, and the line never really makes a sensible prediction for any distance.

The solution is to not fit a straight line. Rather, we will use a transformation of the linear model. Below, we introduce a broad outline of the use and some of the properties of such a model.

```
distance<- c( 22, 120, 400 ,102,22, 620, 100 ,202,222, 130, 800 ,852)

RealEstateAgent <- c("show", "show", "don't show", "show", "show",
                    "don't show", "don't show", "show", "show",
                    "don't show", "don't show", "don't show")

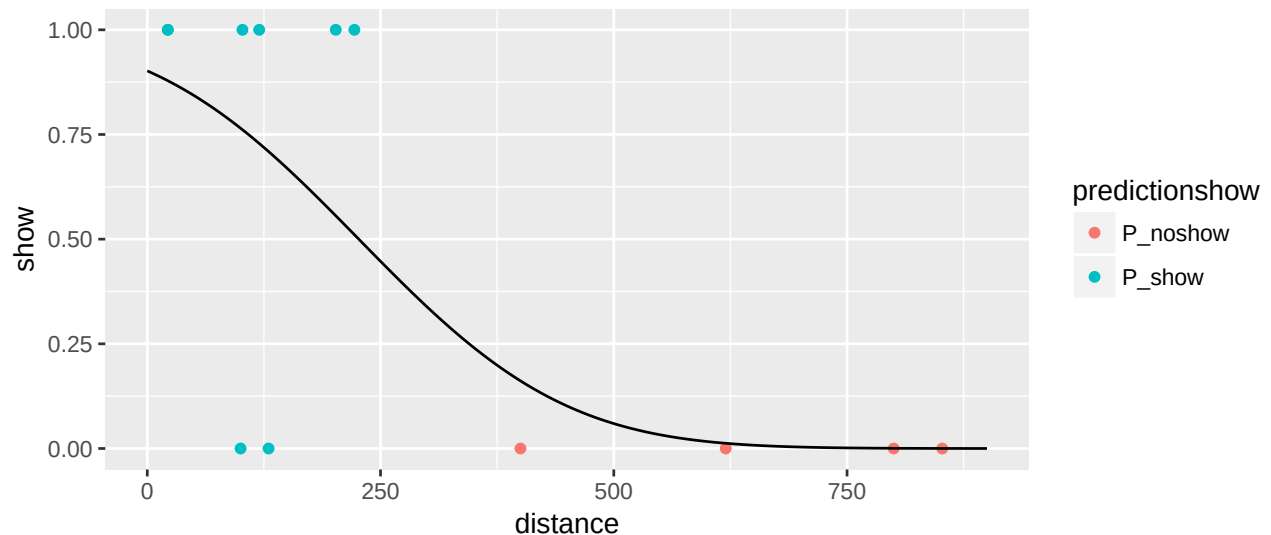
show <- ifelse(RealEstateAgent=="show", 1 , 0)

modshowprobit <- glm(show~distance, family=binomial(link="probit"))

modpredictions <- data.frame(distance, show,
                             predictionshow = ifelse(predict(modshowprobit, type="response") >.5,
                                                         "P_show", "P_noshow"))

simulatelatent <- data.frame(distance=0:900,
                             predictmod = predict(modshowprobit,
                                                    newdata=data.frame(distance=0:900),
                                                    type="response"))

ggplot(modpredictions, aes(distance, show) )+
  geom_point(aes( color=predictionshow))+
  geom_line(data=simulatelatent,
            aes(distance, predictmod))
```



## Probit for categorical dependent variables

In the case of the reliability of the religion measurements, the dependent variable is binary, the independent variable is a 3 category vector, which we can indicate as follows:

```
probitmod <-  
  glm(bigchange ~ reliab,  
      data = religions,  
      family = binomial(link = "probit"))
```

Implicitly the computer takes `reliab`, and splits it into two category indicators. The first is the omitted reference category, the other two are entered linearly. You could force the model to do the same thing explicitly as well, using the `I(x == "category")`, which takes on 1 if x takes on that category and 0 otherwise:

```
probitmod <-  
  glm(  
    bigchange ~ I(reliab == "Low") + I(reliab == "Medium"),  
    data = religions,  
    family = binomial(link = "probit")  
  )
```

The formula can be described notationally as follows:

$$Pr(y_i = 1|x_i) = \Phi(\alpha + \beta_{low}I(x_i = 'Low') + \beta_{medium}I(x_i = 'Medium'))$$

You read this as the probability that y is equal to 1 given x is equal to the standard normal CDF evaluated at  $\alpha$  plus  $\beta_{low}$  if the observation is low reliability,  $\alpha$  plus  $\beta_{medium}$  if the observation is medium reliability, and just  $\alpha$  if the observation is high reliability. We will learn more about the normal CDF later in this course, but suffice it to say that it is an S or backward S shaped curve. You can explore it in R using the `pnorm()` command.



Just as with a linear model, the coefficients can be returned either directly or in a summary command.

```
coef(probitmod)
```

```
##              (Intercept)      I(reliab == "Low")TRUE
##              -1.715708                NA
## I(reliab == "Medium")TRUE
##              NA
```

```
summary(probitmod)
```

```
##
## Call:
## glm(formula = bigchange ~ I(reliab == "Low") + I(reliab == "Medium"),
##      family = binomial(link = "probit"), data = religions)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -0.2969  -0.2969  -0.2969  -0.2969   2.5076
##
## Coefficients: (2 not defined because of singularities)
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -1.71571    0.04966  -34.55  <2e-16 ***
## I(reliab == "Low")TRUE         NA         NA      NA      NA
## I(reliab == "Medium")TRUE      NA         NA      NA      NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 709.02  on 1994  degrees of freedom
## Residual deviance: 709.02  on 1994  degrees of freedom
## AIC: 711.02
##
## Number of Fisher Scoring iterations: 5
```

The difference is that the coefficients are on the latent, non-linear cumulative normal distribution. To interpret them, we can pass it through R's `cdf` function for the normal.

Suppose we want the probability of a big change when the reliability of the data is high. This is just the intercept,  $\alpha$ , as the other two coefficients are set to 0. Reading the regression table, we see that  $\alpha = -1.61687$ . We pass the value into the `pnorm` function, which is R's calculator for the normal CDF.

```
pnorm(coef(probitmod)[1])
```

```
## (Intercept)
## 0.04310777
```

If the the reliability is medium, then we add the intercept to the coefficient on medium, which is the third element in the `coef` command, and then pass the result to the `pnorm` command:

```
# pnorm(coef(probitmod)[1] + coef(probitmod)[3])
pnorm( - 1.61687 -0.09783)
```

```
## [1] 0.04320012
```

If we compare these to the table of proportions, we see the similarity. Here I round to the nearest fourth digit.

```
round(prop.table(table(religions$reliab, religions$bigchange), margin=1),4)
```

```
##
##      0      1
## 1 0.9470 0.0530
## 2 0.9568 0.0432
## 3 0.9764 0.0236
```

The second row, second column indicates that among medium reliable data, 4.32 percent experience a big change. This is exactly the interpretation we can give to the tranformed coefficient from the probit model. You can read more about these and other related models in J. Scott Long's **Regression Models for Categorical and Limited Dependent Variables** or in a course on Maximum Likelihood Estimation (MLE).

Other than tables, a useful way to present discrete data is with bar charts. Try each of the following commands

```
# ggplot(religions, aes(reliab))+  
# geom_bar()  
  
# ggplot(religions, aes(reliab, group=as.factor(bigchange), fill=as.factor(bigchange)))+  
# geom_bar()  
  
# ggplot(religions, aes(reliab, group=as.factor(bigchange), fill=as.factor(bigchange)))+  
# geom_bar(position="dodge")  
  
ggplot(religions, aes( reliab,  
  group = as.factor(bigchange),  
  fill = as.factor(bigchange))) +  
geom_bar(position = "dodge") +  
guides(fill = guide_legend(title = "Big Changes")) +  
theme(legend.position = "bottom")
```

## Don't know how to automatically pick scale for object of type labelled. Defaulting to continuous.

